INDIVIDUAL MODULE-WISE LAB WORKBOOK

Automation Testing Training

Name	**Pritha Dalui**
Enrollment no	**12023002001256**
Class roll no	**87**
Stream	**Computer Science & Engineering**
Semester	**7th**
Institute name	**Institute of Engineering & Management, Newtown**

Modules Covered

- Module 1 – Automation with Selenium
- Module 2 – Unit Test Frameworks
- Module 3 – Python BDD Restful
- Module 4 – Robot Framework

Table of Contents / Experiment Index

Exp.	Module	Experiment
1	Automation with Selenium	Multi-Locator Challenge – identification using By ID, Name, XPath and CSS
2	Automation with Selenium	Synchronization – Explicit Waits
3	Automation with Selenium	JavaScript Alerts and Prompt Handling
4	Automation with Selenium	HTML Table – Read, Search and Extract Data
5	Automation with Selenium	Windows, Tabs and Iframes
6	Unit Test Frameworks	Data-Driven Testing using Excel/CSV/JSON with unittest
7	Unit Test Frameworks	PyTest Integration, Fixtures and HTML Reporting
8	Unit Test Frameworks	Page Object Model using PyTest
9	Python BDD Restful	BDD Setup and REST API Validation using Python/Behave
10	Python BDD Restful	Test Data Driven API Automation
11	Python BDD Restful	Selenium Page Object Model with Behave
12	Robot Framework	Basic Robot Framework Selenium Automation – Browser Launch and Page Validation
13	Robot Framework	SeleniumLibrary – Input Text and Page Should Contain Element
14	Robot Framework	Variables – Using Variables in Robot Framework Automation
15	Robot Framework	User-Defined Keywords – Creating Reusable Automation Keywords
16	Robot Framework	Setup and Teardown – Suite and Test-Level Setup/Teardown
17	Robot Framework	Data-Driven Testing – Test Templates and Multiple Test Data
18	Robot Framework	Page Object Model – Reusable Page Objects with Robot Framework

Module 1 - Automation with Selenium

Experiment 1: Multi-Locator Challenge – Identify and Interact with Web Elements

Module	Automation with Selenium
Experiment Title	Multi-Locator Challenge – Identify and Interact with Web Elements

1. Problem Statement

Automate a login workflow on a practice web page by locating the username, password and login button using different Selenium locator strategies.

2. Objective

To understand web-element identification and use ID, Name, CSS Selector and XPath locators for reliable browser automation.

3. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- Chrome/Edge browser
- VS Code or PyCharm

Concepts:

- Locators: ID, Name, CSS Selector and XPath
- WebDriver
- send_keys()
- click()
- URL validation

4. Source Code / Implementation Steps

1. Install Selenium using pip and verify the installation.
2. Open the practice application in a browser.
3. Inspect the username, password and login button elements.
4. Create locators using different strategies.
5. Enter valid/invalid data and click Login.
6. Assert that the resulting URL or page text is as expected.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.maximize_window()

try:
    driver.get("https://www.saucedemo.com/")
    wait = WebDriverWait(driver, 10)

        username = wait.until(
```

```
        EC.visibility_of_element_located(
            (By.ID, "user-name")
        )
    )

    username.send_keys("standard_user")

        password = driver.find_element(
        By.NAME,
        "password"
    )

    password.send_keys("secret_sauce")

    login_button = driver.find_element(
        By.CSS_SELECTOR,
        "input[type='submit']"
    )

    login_button.click()

    products = wait.until(
        EC.visibility_of_element_located(
            (By.XPATH, "//span[text()='Products']")
        )
    )

    print("Login Successful")
    print("Page Title:", driver.title)
    print("Products Text:", products.text)
    print("Current URL:", driver.current_url)
finally:

    driver.quit()
```

5. Output and Screenshots

```
PS D:\>  & 'C:\Users\SAMADRITA HAZRA\AppData\Local\Python\pythoncore-3.14-64\python.exe'
  '65421' '--' 'D:\exp01_multi_locator.py'
Login Successful
Page Title: Swag Labs
Products Text: Products
Current URL: https://www.saucedemo.com/inventory.html
```

6. Result

The login workflow was automated successfully using Selenium locators.

7. Observation

Different locator strategies can identify the same or related web elements; stable attributes should be preferred where available.

8. Conclusion

Selenium WebDriver can reliably automate basic browser interactions when correct locators are selected.

Experiment 2: Synchronization – Explicit Waits

Module	Automation with Selenium
Experiment Title	Synchronization – Explicit Waits

1. Problem Statement
Automate an interaction with an element that becomes available or clickable after a page action, without using fixed sleep delays.

2. Objective
To understand synchronization and implement WebDriverWait with an expected condition.

3. Tools, Software and Concepts Used
- Python 3
- Selenium WebDriver
- Chrome/Edge
- VS Code

Concepts:
- Explicit wait
- Expected Conditions
- element_to_be_clickable
- presence_of_element_located
- Timeout handling

4. Source Code / Implementation Steps
1. Open the target application.
2. Identify an element that is loaded dynamically.
3. Create a WebDriverWait object with a suitable timeout.
4. Wait for the element condition to become true.
5. Interact with the element.
6. Run the test repeatedly to confirm stable behavior.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.maximize_window()

try:

    driver.get(
        "https://the-internet.herokuapp.com/dynamic_loading/1"
    )

    start_button = driver.find_element(
        By.XPATH,
```

```
        "//button[text()='Start']"
    )

    start_button.click()

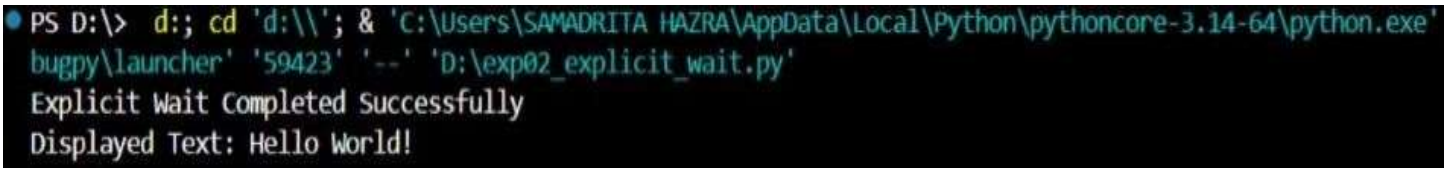
    wait = WebDriverWait(driver, 15)

    hello_text = wait.until(
        EC.visibility_of_element_located(
            (By.XPATH, "//div[@id='finish']/h4")
        )
    )

    print("Explicit Wait Completed Successfully")
    print("Displayed Text:", hello_text.text)

finally:
    driver.quit()
```

5. Output and Screenshots



6. Result

The element was accessed using an explicit wait without relying on time.sleep().

7. Observation

Explicit waits synchronize the test with actual application state and reduce timing-related failures.

8. Conclusion

Explicit waits are suitable when an element may take variable time to become ready.

Experiment 3: JavaScript Alerts and Prompt Handling

Module	Automation with Selenium
Experiment Title	JavaScript Alerts and Prompt Handling

1. Problem Statement
Handle JavaScript alert, confirmation and prompt dialogs generated by a web application.

2. Objective
To automate browser dialogs using Selenium's Alert interface.

3. Tools, Software and Concepts Used
- Python 3
- Selenium WebDriver
- JavaScript alerts
- switch_to.alert

Concepts:
- Alert
- Confirm
- Prompt
- accept()
- dismiss()
- send_keys()

4. Source Code / Implementation Steps

1. Open a page containing JavaScript dialog controls.
2. Trigger an alert and switch to it.
3. Read the alert text and accept it.
4. Trigger a confirmation dialog and test accept/dismiss.
5. Trigger a prompt, enter text and accept it.
6. Capture the final page state.

```
from selenium import webdriver

from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

driver.maximize_window()

try:

   driver.get(

driver.find_element(

      By.XPATH,
```

```
    "//button[text()='Click for JS Alert']"
).click()
alert = driver.switch_to.alert
print("Alert Message:")
print(alert.text)
alert.accept()
driver.find_element(
    By.XPATH,
    "//button[text()='Click for JS Confirm']"
).click()
confirm = driver.switch_to.alert
print("Confirm Message:")
print(confirm.text)
confirm.dismiss()
result = driver.find_element(
    By.ID,
    "result"
)
print("Confirm Result:")
print(result.text)
driver.find_element(
    By.XPATH,
    "//button[text()='Click for JS Prompt']"
).click()
prompt = driver.switch_to.alert
print("Prompt Message:")
print(prompt.text)
prompt.send_keys("Hello Selenium")
prompt.accept()
```

```
    result = driver.find_element(
        By.ID,
        "result"
    )
    print("Prompt Result:")
    print(result.text)
finally:
    driver.quit()
```

5. Output and Screenshots



6. Result

The JavaScript dialog was detected, read and handled successfully.

7. Observation

Selenium provides switch_to.alert for browser-level JavaScript dialogs.

8. Conclusion

Alert handling is required whenever modal JavaScript dialogs interrupt normal browser interaction.

Experiment 4: HTML Table – Read, Search and Extract Data

Module	Automation with Selenium
Experiment Title	HTML Table – Read, Search and Extract Data

1. Problem Statement
Read data from an HTML table, locate a required row by matching a value, and retrieve the corresponding cell.

2. Objective
To practice dynamic XPath and extraction of information from HTML tables.

3. Tools, Software and Concepts Used
- Python 3
- Selenium WebDriver
- XPath
- HTML tables

Concepts:
- Dynamic XPath
- Rows and cells
- find_elements()
- Text extraction

4. Source Code / Implementation Steps
1. Open the page containing the table.
2. Locate all table rows.
3. Read each row's cell values.
4. Compare the required value with the row data.
5. Print or validate the matching row.
6. Record the output in the workbook.

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()

try:
    driver.get(
        "https://the-internet.herokuapp.com/tables"
    )

    rows = driver.find_elements(
        By.XPATH,

        "//table[@id='table1']/tbody/tr"
    )
```

```
    print("HTML TABLE DATA")
    print("-" * 70)

    for row_number, row in enumerate(rows, start=1):

        cells = row.find_elements(
            By.TAG_NAME,
            "td"
        )
        values = []

        for cell in cells:

            values.append(cell.text)

        print("Row", row_number, ":", values)

    search_name = "Smith"

    print("\nSearching for:", search_name)
    for row in rows:

        cells = row.find_elements(
            By.TAG_NAME,
            "td"
        )

        if cells[0].text == search_name:
            print("Matching Row Found")
            print("Name:", cells[0].text)
            print("Last Name:", cells[1].text)
            print("Email:", cells[2].text)

            break

finally:
    driver.quit()
```

5. Output and Screenshots:

```
PS D:\>  d:; cd 'd:\\'; & 'C:\Users\SAMADRITA HAZRA\AppData\Local\Python\pythoncore-3.14-64\python.exe'
bugpy\launcher' '63721' '--' 'D:\exp04_html_table.py'
HTML TABLE DATA
----------------------------------------------------------------------
Row 1 : ['Smith', 'John', 'jsmith@gmail.com', '$50.00', 'http://www.jsmith.com', 'edit delete']
Row 2 : ['Bach', 'Frank', 'fbach@yahoo.com', '$51.00', 'http://www.frank.com', 'edit delete']
Row 3 : ['Doe', 'Jason', 'jdoe@hotmail.com', '$100.00', 'http://www.jdoe.com', 'edit delete']
Row 4 : ['Conway', 'Tim', 'tconway@earthlink.net', '$50.00', 'http://www.timconway.com', 'edit delete']

Searching for: Smith
Matching Row Found
Name: Smith
Last Name: John
Email: jsmith@gmail.com
```

6. Result

The table rows and cell values were read successfully and the required data could be identified.

7. Observation

XPath can be used to navigate from a table to its rows and cells.

8. Conclusion

Table automation becomes reusable when row/cell selection is based on data instead of hard-coded positions.

Experiment 5: Windows, Tabs and Iframes

Module	Automation with Selenium
Experiment Title	Windows, Tabs and Iframes

1. Problem Statement
Automate a web application containing multiple browser windows/tabs and an iframe, switching between contexts and returning to the main page.

2. Objective
To understand window handles and iframe switching in Selenium.

3. Tools, Software and Concepts Used
- Python 3
- Selenium WebDriver
- window_handles
- switch_to.frame()
- switch_to.window()

Concepts:
- Browser window context
- Window handles
- Iframe
- Frame context switching

4. Source Code / Implementation Steps
1. Open a page containing an iframe and/or multiple windows.
2. Store the main window handle.
3. Switch to the iframe and interact with an element.
4. Switch back to default_content().
5. Open or detect another browser window.
6. Switch using its window handle and perform the required action.
7. Return to the main window.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException

driver = webdriver.Chrome()
driver.maximize_window()

wait = WebDriverWait(driver, 15)

try:
    driver.get(
        "https://the-internet.herokuapp.com/iframe"
```

```
)

print("Main page opened")

# Wait for iframe
iframe = wait.until(
    EC.presence_of_element_located(
        (By.ID, "mce_0_ifr")
    )
)

driver.switch_to.frame(iframe)
print("Switched to iframe")

editor = wait.until(
    EC.presence_of_element_located(
        (By.TAG_NAME, "body")
    )
)

print("Editor body found")

driver.execute_script(
    "arguments[0].focus();",
    editor
)
print("Editor focused")

editor.send_keys(
    Keys.CONTROL,
    "a"
)
    editor.send_keys(
    Keys.BACKSPACE
)
editor.send_keys(
    "Hello from Selenium iframe automation"
)

print(
    "Text entered inside iframe"
)
driver.switch_to.default_content()

print(
    "Returned to main document"
)
driver.get(
    "https://the-internet.herokuapp.com/windows"
)
print(
    "Windows page opened"
)
main_window = driver.current_window_handle
```

```
print(
    "Main Window ID:",
    main_window
)

driver.find_element(
    By.LINK_TEXT,
    "Click Here"
).click()

wait.until(
    EC.number_of_windows_to_be(2)
)

all_windows = driver.window_handles

print(
    "Number of Windows:",
    len(all_windows)
)
    for window in all_windows:
    if window != main_window:

        driver.switch_to.window(
            window
        )
        print(
            "Switched to new window"
        )
        print(
            "New Window Title:",
            driver.title
        )

        heading = wait.until(
            EC.visibility_of_element_located(
                (By.TAG_NAME, "h3")
            )
        )
        print(
            "New Window Heading:",
            heading.text
        )
        driver.close()

        print(
            "New window closed"
        )
        break
driver.switch_to.window(
    main_window
)
print(
    "Returned to main window"
```

```
    )
    print(
        "Main Window Title:",
        driver.title
    )

except TimeoutException as error:

    print(
        "Timeout occurred:"
    )
    print(
        repr(error)
    )

finally:
    driver.quit()

    print(
        "Browser closed"

    )
```

5. Output and Screenshots

```
PS D:\>  d:; cd 'd:\\'; & 'C:\Users\SAMADRITA HAZRA\AppData\Local\Python\pythoncore-3.14-64\python.exe'
bugpy\launcher' '60831' '--' 'D:\exp05_windows_iframes.py'
Main page opened
Switched to iframe
Editor body found
Editor focused
Text entered inside iframe
Returned to main document
Windows page opened
Main Window ID: 0C3481AB5F70D089BFA96DF3F6AFEFD0
Number of Windows: 2
Switched to new window
New Window Title: New Window
New Window Heading: New Window
New window closed
Returned to main window
Main Window Title: The Internet
Browser closed
```

6. Result

The required frame/window context was selected and the interaction was completed.

7. Observation

Selenium commands operate in the current browsing context, so explicit switching is required.

8. Conclusion

Window handles and frame switching are essential for automating applications with multiple browsing contexts.

MODULE 2 — UNIT TEST FRAMEWORKS
Experiment 6: Data-Driven Testing using unittest

Module	Unit Test Frameworks
Experiment Title	Data-Driven Testing using unittest

1. Problem Statement
Execute the same test scenario with multiple sets of input data and verify the expected results.

2. Objective
To understand data-driven test design and reduce duplicate test code.

3. Tools, Software and Concepts Used
- Python 3
- unittest
- CSV/Excel/JSON test data
- Selenium (if UI test is used)

Concepts:
- TestCase
- setUp()
- tearDown()
- data-driven execution
- assertions

4. Source Code / Implementation Steps
7. Prepare a test-data file containing multiple input combinations and expected results.
8. Create a unittest TestCase class.
9. Load the data in the test setup.
10. Loop through each data row and execute the same test logic.
11. Use assertions to compare actual and expected values.
12. Run the complete suite and record the result.

```
import unittest
class LoginDataDrivenTest(unittest.TestCase):

    def setUp(self):
        print("\nSetting up test data")
        self.test_data = [
            ("standard_user", "secret_sauce", True),
            ("wrong_user", "wrong_password", False),
            ("standard_user", "wrong_password", False)

    def login_check(self, username, password):
        if username == "standard_user" and password == "secret_sauce":
            return True
        return False
    def test_login_with_multiple_data(self):
```

```
        for username, password, expected in self.test_data:

            with self.subTest(
                username=username,
                password=password
            ):
                actual = self.login_check(
                    username,
                    password
                )
                self.assertEqual(
                    actual,
                    expected
                )

    def tearDown(self):
        print("Test completed")

if __name__ == "__main__":
    unittest.main(
        verbosity=2
    )
```

5. Output and Screenshots

```
PS D:\>  d:; cd 'd:\'; & 'C:\Users\SAMADRITA HAZRA\AppData\Local\Python\pythoncore-3.14-64\python.e
bugpy\launcher' '52790' '--' 'D:\exp06_unittest_data_driven.py'
test_login_with_multiple_data (__main__.LoginDataDrivenTest.test_login_with_multiple_data) ...
Setting up test data
Test completed
ok

----------------------------------------------------------------------
Ran 1 test in 0.001s

OK
```

6. Result

The same test logic was executed for multiple data combinations and validated with assertions.

7. Observation

Data-driven testing improves reuse and makes it easier to expand test coverage.

8. Conclusion

unittest can provide a structured foundation for repeatable automated test execution.

Experiment 7: PyTest Integration, Fixtures and HTML Reporting

Module	Unit Test Frameworks
Experiment Title	PyTest Integration, Fixtures and HTML Reporting

1. Problem Statement
Create PyTest tests with fixtures, run multiple test cases and generate an HTML execution report.

2. Objective
To learn PyTest structure, fixtures, assertions, test discovery and reporting.

3. Tools, Software and Concepts Used
- Python 3
- PyTest
- pytest-html
- Selenium WebDriver (optional)

Concepts:
- test_*.py naming
- Fixtures
- Assertions
- Parameterized/data-driven tests
- HTML report

4. Source Code / Implementation Steps
1. Install pytest and pytest-html.
2. Create a test file named using the PyTest naming convention.
3. Create a fixture for setup/cleanup.
4. Write one or more test functions.
5. Run pytest from the terminal.
6. Generate an HTML report using pytest-html.
7. Open the report and verify pass/fail details.

```
import pytest
@pytest.fixture
def test_data():

    print("\nSetting up test data")

    data = {
        "username": "standard_user",
        "password": "secret_sauce",
        "title": "Swag Labs"
    }

    yield data

    print("\nCleaning up")
```

```
def test_username(test_data):
    assert test_data["username"] == "standard_user"


def test_password(test_data):
    assert test_data["password"] == "secret_sauce"


def test_title(test_data):
    assert test_data["title"] == "Swag Labs"

def test_invalid_password(test_data):
    wrong_password = "wrong_password"
    assert wrong_password != test_data["password"]
```

5. Output and Screenshots



report.html

Report generated on 23-Sep-2026 at 22:01:40 by pytest-html v4.2.0

Environment

Python	3.14.2
Platform	Windows-11-10.0.26200-SP0
Packages	• pytest: 9.1.1 • pluggy: 1.6.0
Plugins	• html: 4.2.0 • metadata: 3.1.1

Summary

4 tests took 113 ms.

(Un)check the boxes to filter the results.

0 Failed, 4 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns 0 Retried, Show all details / Hide all details

Result	Test	Duration	Links
Passed	test_exp07_pytest.py::test_username	1 ms	
Passed	test_exp07_pytest.py::test_password	2 ms	
Passed	test_exp07_pytest.py::test_title	2 ms	
Passed	test_exp07_pytest.py::test_invalid_password	1 ms	

6. Result

PyTest tests executed successfully and an HTML execution report was generated.

7. Observation

Fixtures centralize setup/cleanup and reduce repeated code across tests.

8. Conclusion

PyTest provides a concise and extensible framework for automated test execution and reporting.

Experiment 8: Page Object Model using PyTest

Module	Unit Test Frameworks
Experiment Title	Page Object Model using PyTest

1. Problem Statement
Refactor a UI automation test by separating page locators/actions from test assertions using the Page Object Model.

2. Objective
To create maintainable automation by keeping locators and page actions inside page classes.

3. Tools, Software and Concepts Used
- Python 3
- PyTest
- Selenium WebDriver
- Page Object Model

Concepts:
- BasePage
- Page class
- Locators
- Reusable methods
- Assertions in tests

4. Source Code / Implementation Steps
1. Create a BasePage class for common browser operations.
2. Create a LoginPage class containing page locators and login actions.
3. Create a test class/function that uses the page object.
4. Keep assertions in the test layer and page interactions in page classes.
5. Execute the test with PyTest.

conftest.py
```
import pytest

from selenium import webdriver

@pytest.fixture

def driver():

    browser = webdriver.Chrome()

    browser.maximize_window()

    yield browser

    browser.quit()
```

pages/login_page.py

```
from selenium.webdriver.common.by import By

from selenium.webdriver.support.ui import WebDriverWait

from selenium.webdriver.support import expected_conditions as EC

class LoginPage:

    URL = "https://www.saucedemo.com/"

    USERNAME = (

        By.ID,

        "user-name"

    )

    PASSWORD = (

        By.ID,

        "password"

    )

    LOGIN_BUTTON = (

        By.ID,

        "login-button"

    )

    PRODUCTS_TITLE = (

        By.XPATH,

        "//span[text()='Products']"

    )

    def __init__(self, driver):

        self.driver = driver

        self.wait = WebDriverWait(

            driver,

            10

        )

    def open(self):
```

```
        self.driver.get(
            self.URL
        )
    def enter_username(self, username):
        self.driver.find_element(
            *self.USERNAME
        ).send_keys(username)
    def enter_password(self, password):
        self.driver.find_element(
            *self.PASSWORD
        ).send_keys(password)
    def click_login(self):
        self.driver.find_element(
            *self.LOGIN_BUTTON
        ).click()
    def login(self, username, password):
        self.enter_username(username)
        self.enter_password(password)
        self.click_login()
    def get_products_title(self):
        element = self.wait.until(
            EC.visibility_of_element_located(
                self.PRODUCTS_TITLE
            )
        )
        return element.text
```

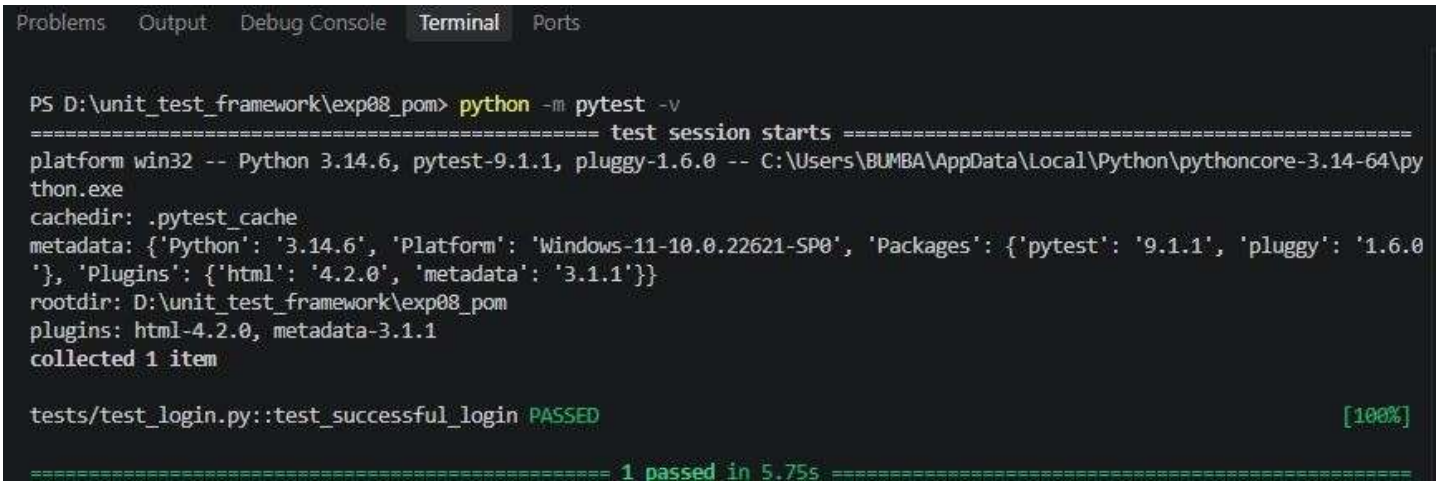
tests/test_login.py

```
from pages.login_page import LoginPage

def test_successful_login(driver):
```

```
    login_page = LoginPage(driver)
    login_page.open()
    login_page.login(
        "standard_user",
        "secret_sauce"
    )
    assert (
        login_page.get_products_title()
        == "Products"
    )
```

5. Output and Screenshots



6. Result

The UI test was reorganized using the Page Object Model and executed successfully.

7. Observation

Separating page actions from tests makes locator maintenance easier.

8. Conclusion

POM provides a reusable structure for scalable Selenium automation.

Experiment 9: BDD Setup and REST API Validation using Python/Behave

Module	Python BDD Restful
Experiment Title	BDD Setup and REST API Validation using Python/Behave

1. Problem Statement

Create a Behavior-Driven Development scenario for an API request and validate the HTTP response.

2. Objective

To understand BDD feature files, step definitions, API requests, status codes and JSON response validation.

3. Tools, Software and Concepts Used

- Python 3
- Behave
- Requests
- PyCharm/VS Code
- REST API

Concepts:

- REST vs SOAP
- HTTP GET/POST
- JSON
- Status codes
- BDD Given/When/Then

4. Source Code / Implementation Steps

1. Install behave and requests.
2. Create a feature file containing a business-readable scenario.
3. Create Python step definitions.
4. Send an API request using requests.
5. Validate status code and selected JSON fields.
6. Run the Behave suite and record the scenario result.

features/api.feature

```
Feature: REST API validation

  Scenario: Validate a GET request
    Given the API endpoint is available
    When I send a GET request
    Then the response status code should be 200
    And the response should contain userId
```

features/steps/api_steps.py

```
import requests

from behave import given
from behave import when
from behave import then
```

```
@given("the API endpoint is available")
def step_endpoint(context):

    context.url = (
        "https://jsonplaceholder.typicode.com/posts/1"
    )
@when("I send a GET request")
def step_get_request(context):

    context.response = requests.get(
        context.url,
        timeout=10
    )
@then("the response status code should be 200")
def step_status_code(context):

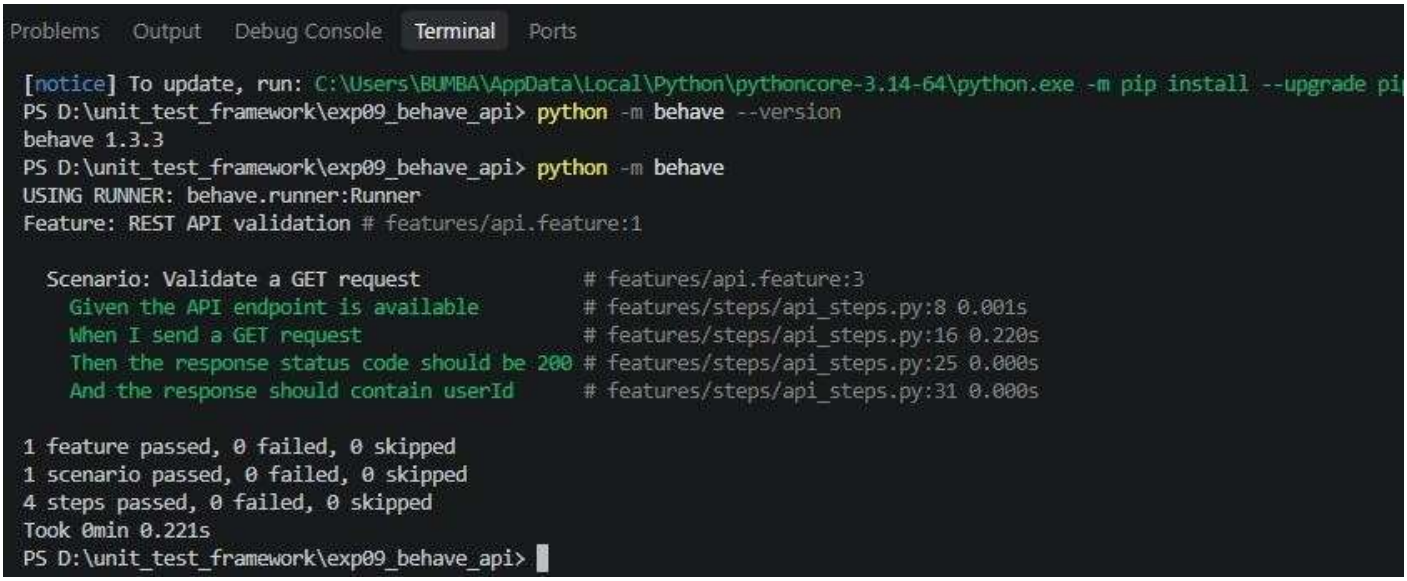
    assert context.response.status_code == 200

@then("the response should contain userId")
def step_response_json(context):

    data = context.response.json()
    assert "userId" in data

    print("\nResponse JSON:")
    print(data)
```

5. Output and Screenshots



6. Result

The REST API scenario was executed and the expected HTTP response was validated.

7. Observation

BDD separates readable business behavior from the underlying automation implementation.

8. Conclusion

Behave and Requests can be combined to build readable and reusable API automation.

Experiment 10: Test Data Driven API Automation

Module	Python BDD Restful
Experiment Title	Test Data Driven API Automation

1. Problem Statement
Execute the same API behavior with multiple test-data values and validate the response for every input.

2. Objective
To automate API scenarios using reusable test data instead of hard-coded values.

3. Tools, Software and Concepts Used
- Python 3
- Behave
- Requests
- CSV/JSON test data
- REST API

Concepts:
- Data-driven testing
- Session management
- Headers
- Payload
- PUT/PATCH/DELETE
- Response validation

4. Source Code / Implementation Steps
1. Prepare reusable API test data in JSON or another suitable format.
2. Read the data from the test-data source.
3. Create a reusable API request method.
4. Send requests using the supplied payload/parameters.
5. Validate status codes and response fields.
6. Execute all data combinations and record the result.

features/api_data.feature
```
Feature: Data driven REST API testing

  Scenario Outline: Validate different post IDs
    Given the API URL for post "<post_id>"
    When I send a GET request for the post
    Then the response status code should be <status_code>

    Examples:

      | post_id | status_code |
      | 1       | 200         |
      | 2       | 200         |
      | 3       | 200         |
      | 4       | 200         |
```

features/steps/api_data_steps.py

```
import requests

from behave import given

from behave import when

from behave import then


@given('the API URL for post "{post_id}"')
def step_create_url(context, post_id):

    context.url = (
        "https://jsonplaceholder.typicode.com/posts/"
        + post_id
    )


@when("I send a GET request for the post")
def step_send_request(context):

    context.response = requests.get(
        context.url,
        timeout=10
    )

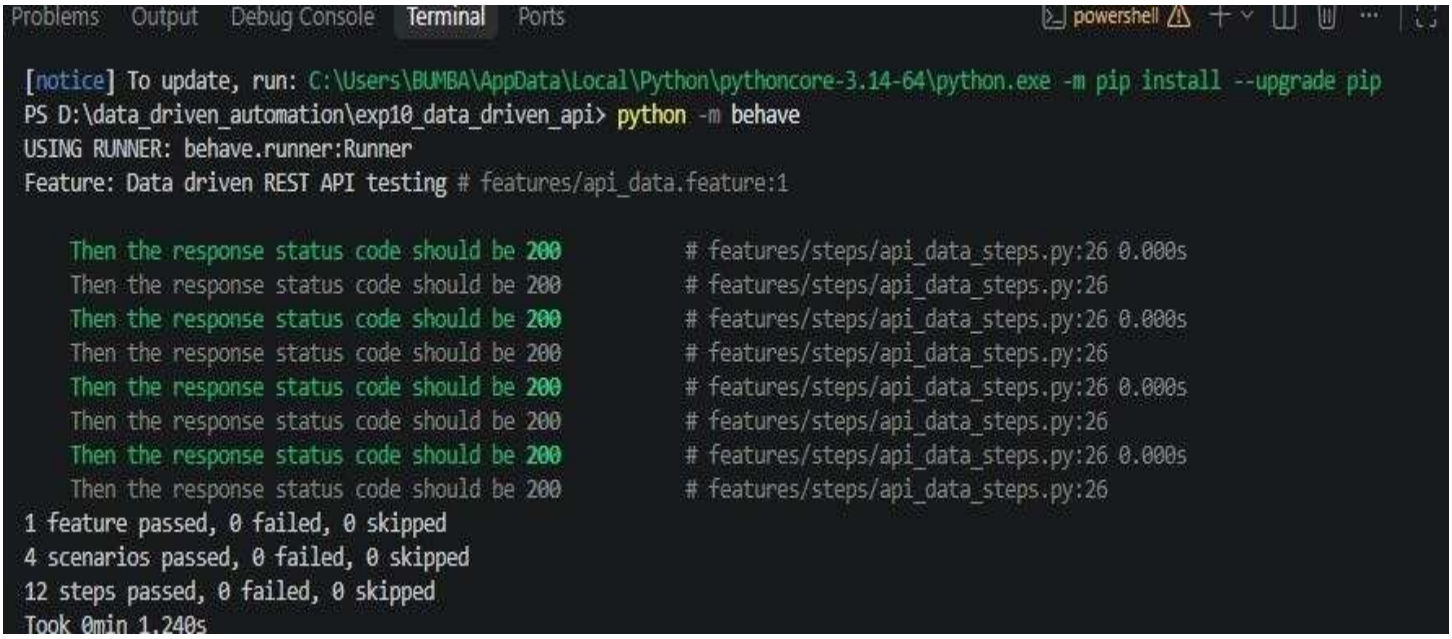

@then(
    "the response status code should be {status_code:d}"
)
def step_check_status(context, status_code):

    assert (
        context.response.status_code
        == status_code
    )

    data = context.response.json()

    print("\nResponse:")
    print(data)
```

5. Output and Screenshots



6. Result

The API test logic was executed against multiple data values and the expected responses were validated.

7. Observation

Reusable test data allows the same API logic to cover more cases with less duplication.

8. Conclusion

Data-driven API automation improves test coverage while keeping the implementation maintainable.

Experiment 11: Selenium Page Object Model with Behave

Module	Python BDD Restful
Experiment Title	Selenium Page Object Model with Behave

1. Problem Statement

Combine Selenium UI automation, Page Object Model and Behave to automate a web workflow using business-readable BDD scenarios.

2. Objective

To integrate UI automation with BDD and POM so that feature files describe behavior while page classes contain UI implementation details.

3. Tools, Software and Concepts Used

- Python 3
- Behave
- Selenium WebDriver
- Page Object Model
- Gherkin

Concepts:

- Feature file
- Step definitions
- Page objects
- Given/When/Then
- Selenium locators

4. Source Code / Implementation Steps

1. Create a feature file describing the user workflow.
2. Create page-object classes for the required pages.
3. Create step definitions that call page-object methods.
4. Configure the browser setup and teardown.
5. Run the Behave feature.
6. Capture the application result and terminal output.

pages/login_page.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class LoginPage:

    URL = "https://www.saucedemo.com/"

    USERNAME = (
        By.ID,
        "user-name"
    )
    PASSWORD = (
```

```
    By.ID,
    "password"
)

LOGIN_BUTTON = (
    By.ID,
    "login-button"
)

PRODUCTS_TITLE = (
    By.XPATH,
    "//span[text()='Products']"
)

def __init__(self, driver):

    self.driver = driver

    self.wait = WebDriverWait(
        driver,
        10
    )

def open(self):

    self.driver.get(
        self.URL
    )

def enter_username(self, username):

    self.driver.find_element(
        *self.USERNAME
    ).send_keys(username)

def enter_password(self, password):

    self.driver.find_element(
        *self.PASSWORD
    ).send_keys(password)

def click_login(self):

    self.driver.find_element(
        *self.LOGIN_BUTTON
    ).click()

def get_products_title(self):

    element = self.wait.until(
        EC.visibility_of_element_located(
            self.PRODUCTS_TITLE
        )
    )
```

```
        return element.text
```

features/environment.py

```
from selenium import webdriver


def before_scenario(context, scenario):

    context.driver = webdriver.Chrome()

    context.driver.maximize_window()


def after_scenario(context, scenario):

    context.driver.quit()
```

features/login.feature

```
Feature: SauceDemo Login

  Scenario: Successful login

    Given I open the SauceDemo login page

    When I enter valid SauceDemo credentials

    And I click the login button

    Then I should see the Products page
```

features/steps/login_steps.py

```
from behave import given
from behave import when
from behave import then

from pages.login_page import LoginPage


@given("I open the SauceDemo login page")
def step_open(context):

    context.login_page = LoginPage(
        context.driver
    )

    context.login_page.open()


@when("I enter valid SauceDemo credentials")
def step_credentials(context):
```

```
    context.login_page.enter_username(
        "standard_user"
    )

    context.login_page.enter_password(
        "secret_sauce"
    )

@when("I click the login button")
def step_login(context):

    context.login_page.click_login()


@then("I should see the Products page")
def step_products(context):

    assert (
        context.login_page.get_products_title()
        == "Products"
    )
```

5. Output and Screenshots

```
Problems  Output  Debug Console  Terminal  Ports

behave_selenium\\features\\steps'
PS D:\data_driven_automation\exp11_behave_selenium> python -m behave
USING RUNNER: behave.runner:Runner
Feature: SauceDemo Login # features/login.feature:1

  Scenario: Successful login                 # features/login.feature:3
    Given I open the SauceDemo login page    # features/steps/login_steps.py:8 0.169s
    When I enter valid SauceDemo credentials # features/steps/login_steps.py:18 0.457s
    And I click the login button             # features/steps/login_steps.py:30 0.078s
    Then I should see the Products page      # features/steps/login_steps.py:36 0.089s

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
4 steps passed, 0 failed, 0 skipped
Took 0min 0.793s
PS D:\data_driven_automation\exp11_behave_selenium> python -m behave
USING RUNNER: behave.runner:Runner
Feature: SauceDemo Login # features/login.feature:1
```

6. Result

The Selenium workflow was executed through a Behave scenario using the Page Object Model.

7. Observation

BDD improves readability while POM keeps Selenium implementation details organized.

8. Conclusion

Combining Behave, Selenium and POM provides a structured approach for maintainable UI automation.

Module 4: Robot Framework Automation Testing

Experiment 12: Basic Syntax and Keywords in Robot Framework

1. Problem Statement

Automate a basic web interaction in Robot Framework by opening a browser, navigating to a practice website, entering a value in a form field, and verifying that the expected page element is present.

2. Objective

To understand the basic syntax and keyword-driven structure of a Robot Framework test case and use SeleniumLibrary for simple browser automation.

3. Tools, Software and Concepts Used

- Python 3.10
- Robot Framework
- SeleniumLibrary
- Chrome browser / ChromeDriver
- VS Code or similar IDE

Concepts:

- Robot Framework tabular syntax
- Variables and test cases
- SeleniumLibrary keywords
- Element verification
- Input Text and browser automation

4. Source Code / Implementation Steps

1. Create the experiment folder and Robot Framework suite.
2. Import SeleniumLibrary and define the URL, browser, and username variables.
3. Open SauceDemo in Chrome and maximize the browser.
4. Verify the username field with Page Should Contain Element.
5. Enter standard_user using Input Text.
6. Capture browser evidence and close the browser.
7. Review output.xml, log.html, and report.html.

Robot Framework source code:

```
*** Settings ***
Documentation     Experiment 12 - Basic Syntax and Keywords in Robot Framework
Library           SeleniumLibrary

*** Variables ***
${URL}            https://www.saucedemo.com/
${BROWSER}        chrome
${USERNAME}       standard_user

*** Test Cases ***
Verify SauceDemo Login Page Elements
    [Documentation]    Test case to verify basic SeleniumLibrary keywords
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
    Page Should Contain Element    id=user-name
    Input Text    id=user-name    ${USERNAME}
    Capture Page Screenshot    ../../screenshots/exp12/Exp12_Screenshot02_Browser.png
```

Close All Browsers

5. Output and Screenshots

Experiment 12 - Basic Syntax: basic_syntax.robot — Language: Robot Framework

```
*** Settings ***
Documentation     Experiment 12 - Basic Syntax and Keywords in Robot Framework
Library           SeleniumLibrary

*** Variables ***
${URL}            https://www.saucedemo.com/
${BROWSER}        chrome
${USERNAME}       standard_user

*** Test Cases ***
Verify SauceDemo Login Page Elements
    [Documentation]    Test case to verify basic SeleniumLibrary keywords
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
    Page Should Contain Element    id=user-name
    Input Text    id=user-name    ${USERNAME}
    Capture Page Screenshot    ../../screenshots/exp12/Exp12_Screenshot02_Browser.png
    Close All Browsers

```

Fig. 12.1 – Robot Framework source code used for the experiment.

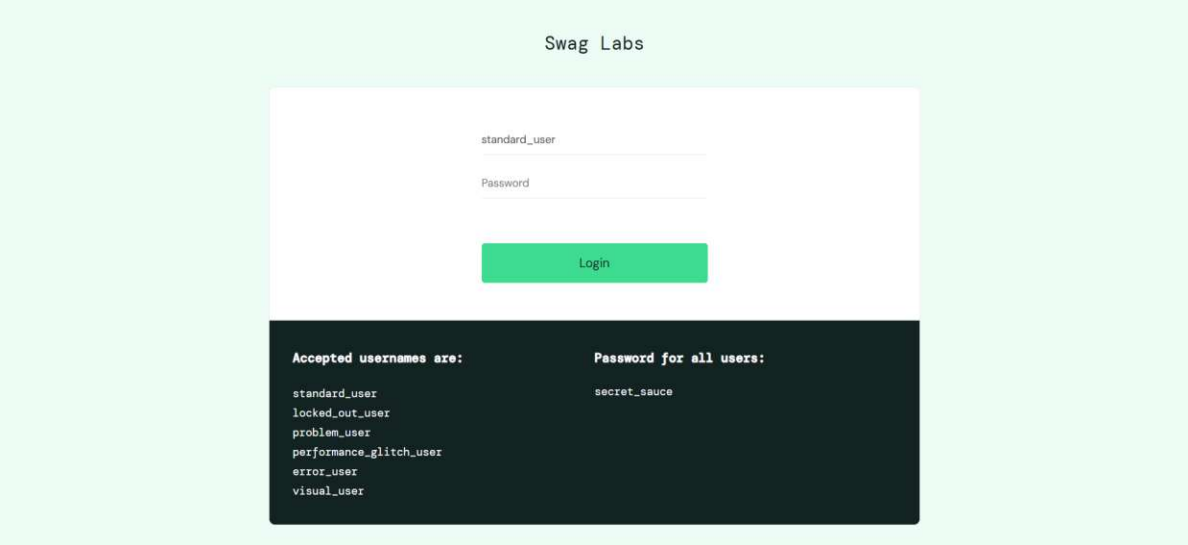


Fig. 12.2 – Browser evidence captured during execution after entering the username.

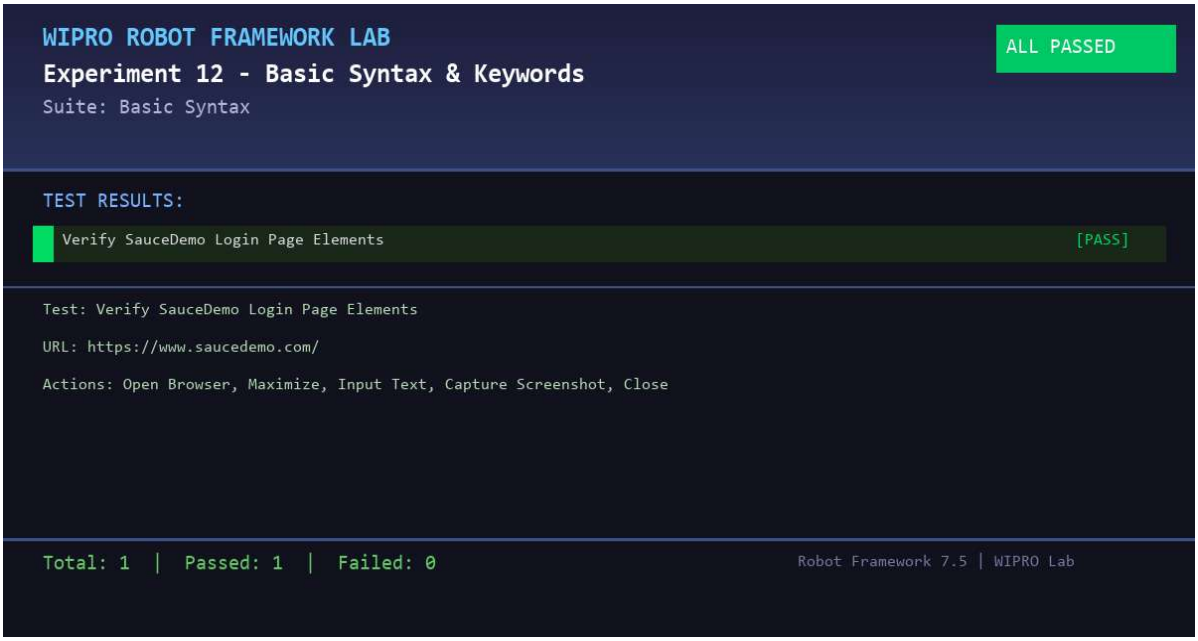


Fig. 12.3 – Robot Framework execution result showing the test passed.

6. Result

The basic syntax test executed successfully with 1 test passed and 0 failures.

7. Observation

- The browser opened successfully and navigated to SauceDemo.
- Page Should Contain Element verified the username field.
- Input Text entered standard_user into the username field.
- Robot Framework generated the execution result files.

8. Conclusion

This experiment showed the basic Robot Framework structure and the use of SeleniumLibrary keywords for browser automation.

Experiment 13: Variables and Data-Driven Automation

1. Problem Statement

Run the same login automation flow with multiple sets of credentials stored in an external CSV file and verify the expected outcome for each data row.

2. Objective

To use Robot Framework variables and DataDriver to implement data-driven automation with multiple input datasets.

3. Tools, Software and Concepts Used

- Python 3.10
- Robot Framework
- SeleniumLibrary
- DataDriver library
- CSV test-data file
- Chrome browser / ChromeDriver
- VS Code or similar IDE

Concepts:

- Scalar variables
- Test Template
- Data-driven testing
- CSV test data
- Parameterized test execution
- Conditional verification

4. Source Code / Implementation Steps

1. Create testdata.csv containing username, password, and expected-result columns.
2. Import SeleniumLibrary and DataDriver.
3. Set Login With Different Credentials as the Test Template.
4. Open SauceDemo and enter the credentials supplied by each CSV row.
5. Verify the expected successful or unsuccessful login condition.
6. Close the browser after each iteration.
7. Review the three generated test iterations and their results.

Robot Framework source code:

```
*** Settings ***
Documentation     Experiment 13 - Variables and Data-Driven Automation
Library           SeleniumLibrary
Library           DataDriver    file=testdata.csv    dialect=excel
Test TemplateLogin With Different Credentials

*** Variables ***
${URL}            https://www.saucedemo.com/
${BROWSER}        chrome

*** Test Cases ***
Login Test with user ${username} and pass ${password}    ${username}    ${password}    ${expected}

*** Keywords ***
Login With Different Credentials
   [Arguments]    ${username}    ${password}    ${expected}
```

```
Open Browser    ${URL}    ${BROWSER}
Maximize Browser Window
Input Text    id=user-name    ${username}
Input Text    id=password    ${password}
Click Button    id=login-button
IF    '${expected}' == 'PASS'
   Page Should Contain Element    id=inventory_container
ELSE
   Page Should Contain Element    xpath=//h3[@data-test='error']
END
Close Browser
```

5. Output and Screenshots

Experiment 13 - Variables & Data-Driven: testdata.csv — Language: CSV Data

```
testdata.csv - Data-Driven Test Data:

${username},${password},${expected}
standard_user,secret_sauce,PASS
wrong_user,wrong_password,FAIL
standard_user,wrong_password,FAIL


Rows represent test iterations:
  Row 1: standard_user / secret_sauce -> PASS (valid login)
  Row 2: wrong_user / wrong_password -> FAIL (invalid - expect error)
  Row 3: standard_user / wrong_password -> FAIL (invalid - expect error)
```

Fig. 13.1 – External CSV test data used for the data-driven test.

Experiment 13 - Variables & Data-Driven: variables_data_driven.robot — Language: Robot Framework

```
*** Settings ***
Documentation     Experiment 13 - Variables and Data-Driven Automation
Library           SeleniumLibrary
Library           DataDriver    file=testdata.csv    dialect=excel
Test Template     Login With Different Credentials

*** Variables ***
${URL}            https://www.saucedemo.com/
${BROWSER}        chrome

*** Test Cases ***
Login Test with user ${username} and pass ${password}    ${username}    ${password}    ${expected}

*** Keywords ***
Login With Different Credentials
    [Arguments]    ${username}    ${password}    ${expected}
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
    Input Text    id=user-name    ${username}
    Input Text    id=password    ${password}
    Click Button    id=login-button
    IF    '${expected}' == 'PASS'
        Page Should Contain Element    id=inventory_container
    ELSE
        Page Should Contain Element    xpath=//h3[@data-test='error']
    END
    Close Browser
```

Fig. 13.2 – Robot Framework data-driven test implementation.



Fig. 13.3 – Execution result showing three data-driven tests passed.

6. Result

Three data-driven test iterations executed successfully: 3 passed and 0 failed.

7. Observation

- DataDriver read the three CSV rows.
- The same test logic was reused for each dataset.
- Valid credentials produced the expected successful-login condition.
- Invalid credentials produced the expected error condition.

8. Conclusion

This experiment showed how CSV data can be used to run the same test with different inputs.

Experiment 14: Custom Keywords and Libraries

1. Problem Statement

Extend Robot Framework with a Python library containing a custom keyword and use built-in Robot Framework libraries for string and mathematical operations.

2. Objective

To understand how Python methods can be exposed as Robot Framework keywords and how built-in libraries can be used in tests.

3. Tools, Software and Concepts Used

- Python 3.10
- Robot Framework
- Custom Python library
- Robot Framework String library
- BuiltIn keywords
- VS Code or similar IDE

Concepts:

- Custom Python keywords
- Library import
- Arguments and return values
- String manipulation
- Evaluate keyword
- Assertions

4. Source Code / Implementation Steps

1. Create CalculatorLibrary.py with an add_numbers method.
2. Import the Python library and String library in the Robot suite.
3. Call Add Numbers with two numeric arguments and verify the result.
4. Use Convert To Upper Case for string processing.
5. Use Evaluate for a mathematical expression and verify the result.
6. Execute the suite and review the result report.

Robot Framework source code:

```
*** Settings ***
Documentation     Experiment 14 - Custom Keywords and Libraries
Library           CalculatorLibrary.py
Library           String

*** Variables ***
${SAMPLE_TEXT}     robot framework lab automation

*** Test Cases ***
Test Custom Python Library Keyword
    [Documentation]    Test custom add_numbers keyword from CalculatorLibrary
    ${sum}=    Add Numbers    10    20
    Should Be Equal As Integers    ${sum}    30

Test BuiltIn And String Functions
    [Documentation]    Test BuiltIn keywords Convert To Upper Case and Evaluate
    ${upper}=    Convert To Upper Case    ${SAMPLE_TEXT}
    Should Be Equal    ${upper}    ROBOT FRAMEWORK LAB AUTOMATION
```

```
    ${result}=    Evaluate    5 * 6 + 2
    Should Be Equal As Integers    ${result}    32
```

Python custom library source code:

```
class CalculatorLibrary:
    """Custom library for mathematical operations in Robot Framework."""

    def add_numbers(self, a, b):
        """Adds two numbers and returns the sum."""
        return int(a) + int(b)
```

5. Output and Screenshots

Experiment 14 - Custom Keywords: CalculatorLibrary.py — Language: Python

```python
class CalculatorLibrary:
    """Custom library for mathematical operations in Robot Framework."""

    def add_numbers(self, a, b):
        """Adds two numbers and returns the sum."""
        return int(a) + int(b)

```

Fig. 14.1 – Python custom library containing the Add Numbers keyword.

Experiment 14 - Custom Keywords: custom_keywords.robot — Language: Robot Framework

```
*** Settings ***
Documentation     Experiment 14 - Custom Keywords and Libraries
Library           CalculatorLibrary.py
Library           String

*** Variables ***
${SAMPLE_TEXT}    robot framework lab automation

*** Test Cases ***
Test Custom Python Library Keyword
    [Documentation]    Test custom add_numbers keyword from CalculatorLibrary
    ${sum}=    Add Numbers    10    20
    Should Be Equal As Integers    ${sum}    30

Test BuiltIn And String Functions
    [Documentation]    Test BuiltIn keywords Convert To Upper Case and Evaluate
    ${upper}=    Convert To Upper Case    ${SAMPLE_TEXT}
    Should Be Equal    ${upper}    ROBOT FRAMEWORK LAB AUTOMATION
    ${result}=    Evaluate    5 * 6 + 2
    Should Be Equal As Integers    ${result}    32

```

Fig. 14.2 – Robot Framework tests using custom and built-in keywords.

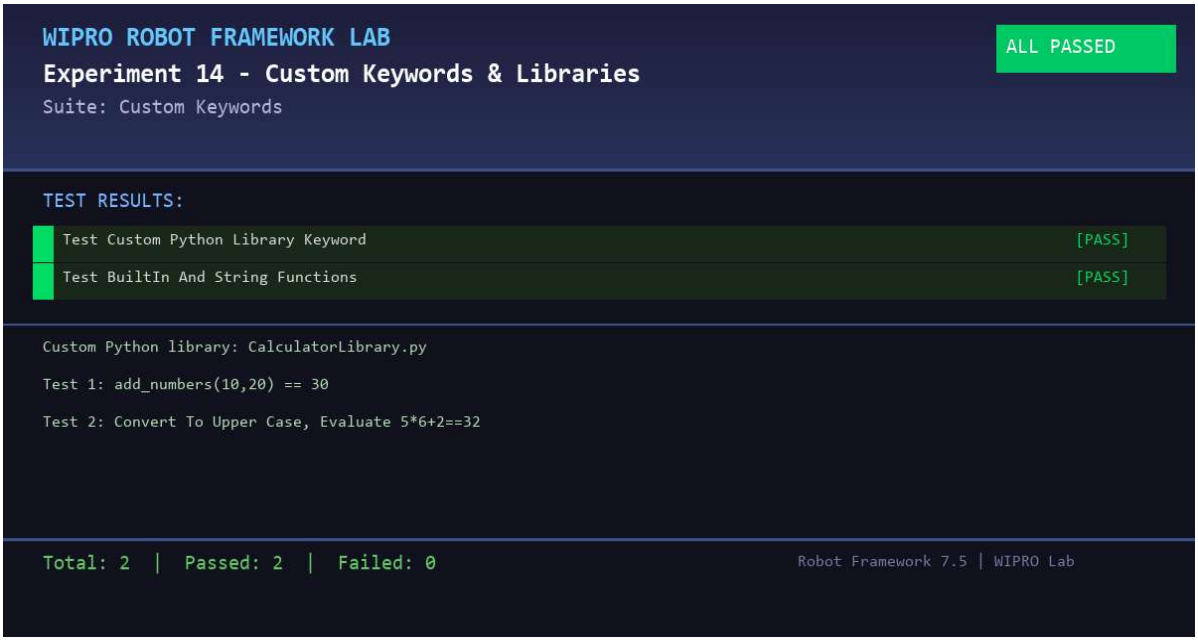


Fig. 14.3 – Execution result showing both tests passed.

6. Result

Both custom-keyword and built-in-library tests passed: 2 passed and 0 failed.

7. Observation

- The Python class was imported successfully as a Robot Framework library.
- Add Numbers returned the expected sum of 30.
- Convert To Upper Case produced the expected uppercase text.
- Evaluate returned the expected value of 32.

8. Conclusion

This experiment showed how a Python function can be used as a Robot Framework keyword and how built-in keywords can be reused.

Experiment 15: Assertions and Verification

1. Problem Statement

Verify expected results after UI and API actions by comparing actual values with expected values using Robot Framework assertions.

2. Objective

To perform explicit UI and REST API verification using Robot Framework assertion keywords.

3. Tools, Software and Concepts Used

- Python 3.10
- Robot Framework
- SeleniumLibrary
- RequestsLibrary
- BuiltIn library
- Chrome browser / ChromeDriver
- JSONPlaceholder REST API

Concepts:

- UI assertions
- API assertions
- Expected vs actual values
- HTTP status verification
- Response-data verification
- Should Be Equal As Strings

4. Source Code / Implementation Steps

1. Open SauceDemo and retrieve the page title.
2. Compare the actual title with the expected title.
3. Create a RequestsLibrary session for JSONPlaceholder.
4. Send a GET request to /posts/1.
5. Verify the response status code is 200.
6. Verify the returned post id is 1 using Should Be Equal As Strings.
7. Review the execution result.

Robot Framework source code:

```
*** Settings ***
Documentation     Experiment 15 - Assertions and Verification
Library           SeleniumLibrary
Library           RequestsLibrary
Library           BuiltIn

*** Variables ***
${UI_URL}         https://www.saucedemo.com/
${BROWSER}        chrome
${API_BASE_URL}   https://jsonplaceholder.typicode.com
${EXPECTED_TITLE}     Swag Labs

*** Test Cases ***
Test A - UI Assertion
    [Documentation]     Verify page title of SauceDemo using UI assertion
    Open Browser    ${UI_URL}     ${BROWSER}
    Maximize Browser Window
```

```
    ${actual_title}=    Get Title
    Should Be Equal    ${actual_title}    ${EXPECTED_TITLE}
    Capture Page Screenshot    ../../screenshots/exp15/Exp15_Screenshot01_UIAssertion.png
    Close Browser
```

Test B - API Assertion

```
    [Documentation]    Send GET request to jsonplaceholder and verify response code and data
    Create Session    apisession    ${API_BASE_URL}
    ${response}=    GET On Session    apisession    /posts/1
    ${status}=    Convert To String    ${response.status_code}
    Should Be Equal As Strings    ${status}    200
    ${body}=    Set Variable    ${response.json()}
    ${post_id}=    Convert To String    ${body['id']}
    Should Be Equal As Strings    ${post_id}    1
```

5. Output and Screenshots

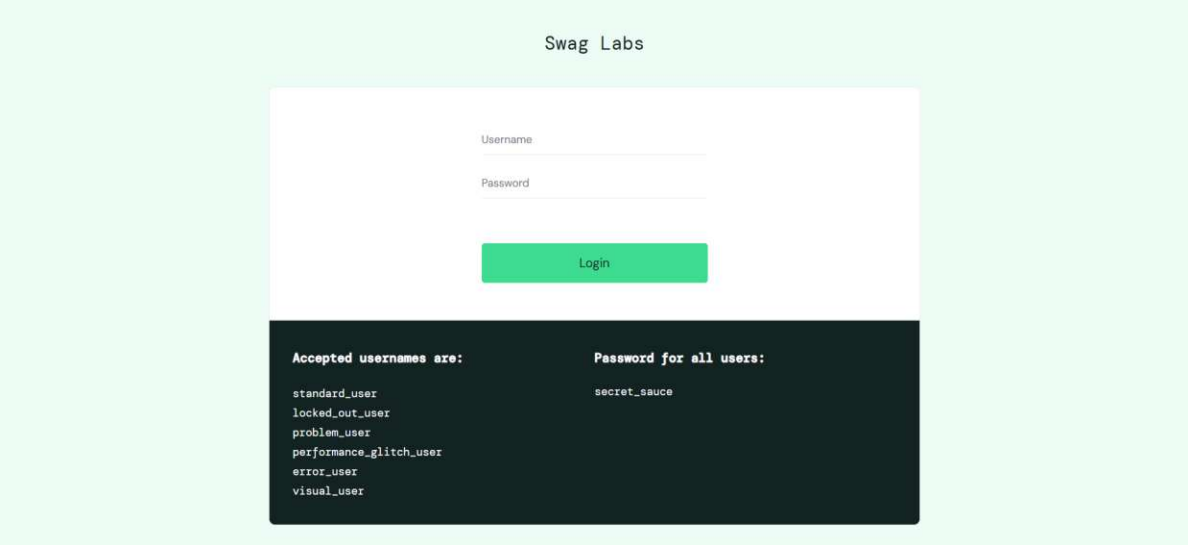


Fig. 15.1 – Browser evidence from the UI assertion test.



Fig. 15.2 – API assertion implementation used to verify the response.

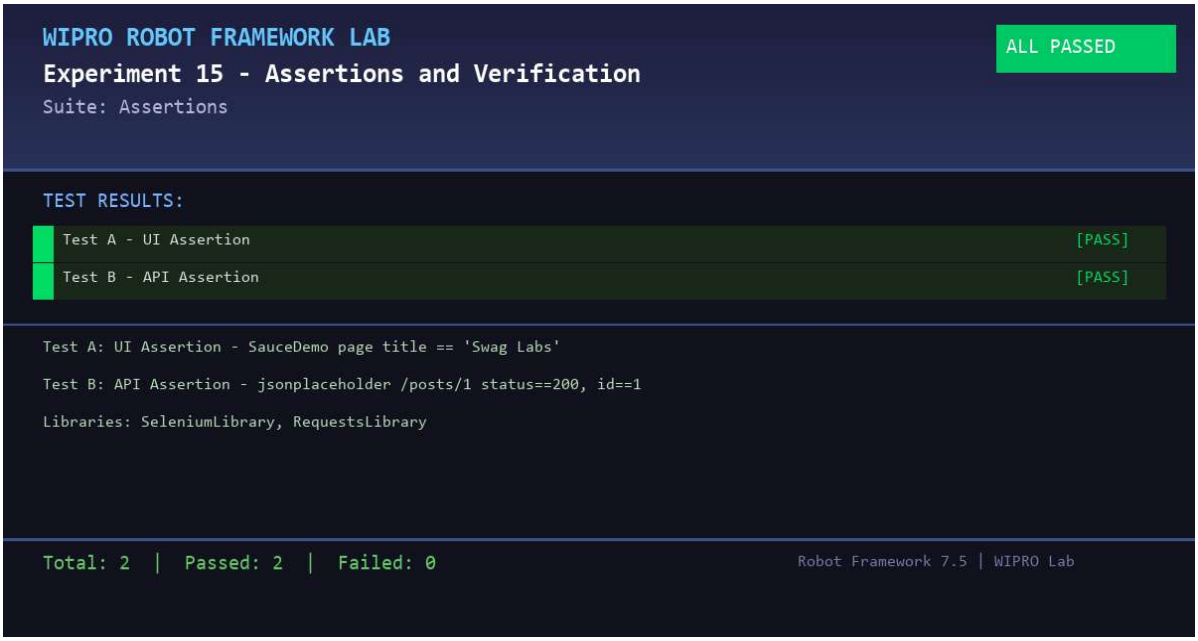


Fig. 15.3 – Execution result showing the UI and API tests passed.

6. Result

The UI and API verification tests passed successfully: 2 passed and 0 failed.

7. Observation

- The SauceDemo page title matched the expected value.
- The API request returned HTTP status 200.
- The response JSON contained post id 1.
- Expected and actual values were compared using Robot Framework assertions.

8. Conclusion

This experiment showed how assertions can be used for both UI and API checks.

Experiment 16: Test Setup and Teardown

1. Problem Statement

Create common setup and teardown actions so that each test begins with a logged-in browser session and ends by releasing the browser resource.

2. Objective

To understand and implement Test Setup and Test Teardown for reusable pre-test and post-test actions.

3. Tools, Software and Concepts Used

- Python 3.10
- Robot Framework
- SeleniumLibrary
- Chrome browser / ChromeDriver
- VS Code or similar IDE

Concepts:

- Test Setup
- Test Teardown
- Reusable keywords
- Browser lifecycle
- Test isolation

4. Source Code / Implementation Steps

1. Define Test Setup as the Login keyword.
2. Define Test Teardown as Logout And Close Browser.
3. Implement the Login keyword to open SauceDemo and sign in.
4. Verify the Products page in one test.
5. Verify the page title in another test.
6. Allow teardown to close the browser after each test.
7. Review the execution result and log.

Robot Framework source code:

```
*** Settings ***
Documentation      Experiment 16 - Test Setup and Teardown
Library            SeleniumLibrary
Test Setup         Login
Test Teardown      Logout And Close Browser

*** Variables ***
${URL}             https://www.saucedemo.com/
${BROWSER}         chrome
${USERNAME}        standard_user
${PASSWORD}        secret_sauce

*** Test Cases ***
Verify Products Page
    [Documentation]    Verify inventory products page is displayed after login
    Page Should Contain Element    xpath=//span[@class='title']
    Capture Page Screenshot    ../../screenshots/exp16/Exp16_Screenshot02_Login.png

Verify Page Title
    [Documentation]    Verify the page title after login
```

```
    ${title}=    Get Title
    Should Be Equal    ${title}    Swag Labs

*** Keywords ***
Login
    [Documentation]    Setup keyword to open browser, navigate to SauceDemo and login
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
    Input Text    id=user-name    ${USERNAME}
    Input Text    id=password    ${PASSWORD}
    Click Button    id=login-button
    Page Should Contain    Products

Logout And Close Browser
    [Documentation]    Teardown keyword to close the active browser instance
    Close Browser
```

5. Output and Screenshots

Experiment 16 - Setup & Teardown: setup_teardown.robot — Language: Robot Framework

```
*** Settings ***
Documentation     Experiment 16 - Test Setup and Teardown
Library           SeleniumLibrary
Test Setup        Login
Test Teardown     Logout And Close Browser

*** Variables ***
${URL}            https://www.saucedemo.com/
${BROWSER}        chrome
${USERNAME}       standard_user
${PASSWORD}       secret_sauce

*** Test Cases ***
Verify Products Page
    [Documentation]    Verify inventory products page is displayed after login
    Page Should Contain Element    xpath=//span[@class='title']
    Capture Page Screenshot    ../../screenshots/exp16/Exp16_Screenshot02_Login.png

Verify Page Title
    [Documentation]    Verify the page title after login
    ${title}=    Get Title
    Should Be Equal    ${title}    Swag Labs

*** Keywords ***
Login
    [Documentation]    Setup keyword to open browser, navigate to SauceDemo and login
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
    Input Text    id=user-name    ${USERNAME}
    Input Text    id=password    ${PASSWORD}
    Click Button    id=login-button
    Page Should Contain    Products

Logout And Close Browser
    [Documentation]    Teardown keyword to close the active browser instance
    Close Browser
```

Fig. 16.1 – Robot Framework source showing Test Setup and Test Teardown.

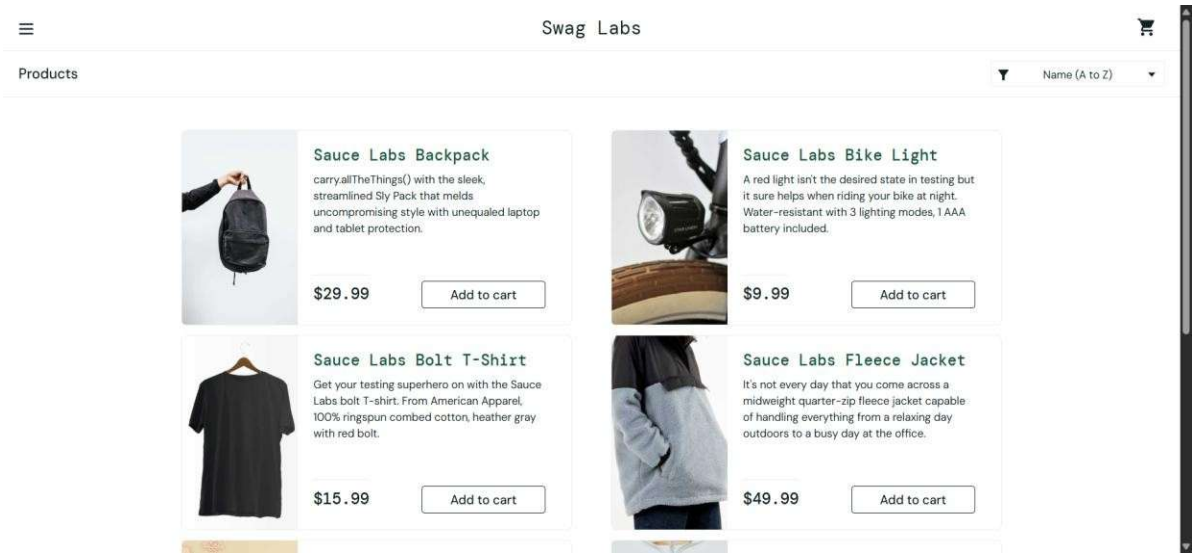


Fig. 16.2 – Browser evidence after the common login setup.

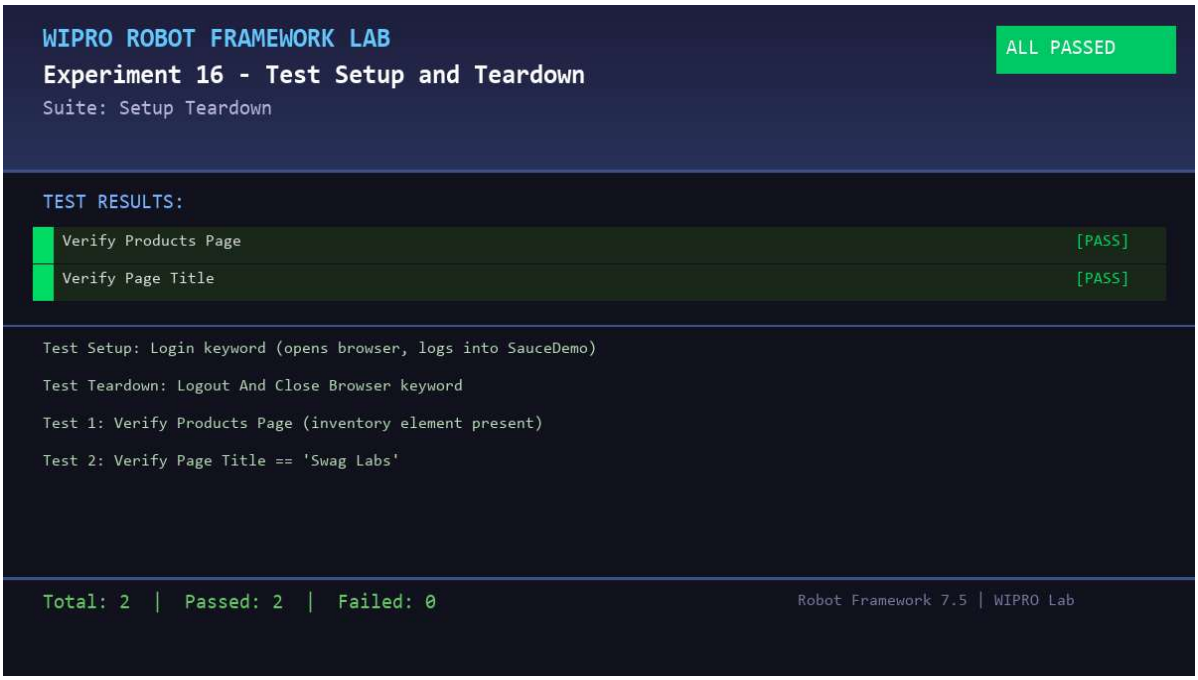


Fig. 16.3 – Execution result showing both tests passed.

6. Result

Both setup/teardown tests passed successfully: 2 passed and 0 failed.

7. Observation

- The Login setup ran before each test.
- The browser opened and the SauceDemo login completed successfully.
- The teardown keyword closed the browser after each test.
- Both verification tests completed with PASS status.

8. Conclusion

This experiment showed how setup and teardown can be used to open and close the browser for each test.

Experiment 17: Tags and Parallel Test Execution

1. Problem Statement

Categorize Robot Framework tests with tags, execute selected groups using include/exclude filters, and demonstrate parallel execution using Pabot.

2. Objective

To use tags for test organization and filtering and to run Robot Framework tests in parallel with Pabot.

3. Tools, Software and Concepts Used

- Python 3.10
- Robot Framework
- SeleniumLibrary
- Pabot / robotframework-pabot
- Chrome browser / ChromeDriver
- VS Code or similar IDE

Concepts:

- Test tags
- --include
- --exclude
- Smoke and regression categories
- Parallel execution
- Pabot processes

4. Source Code / Implementation Steps

1. Create four test cases and assign smoke, regression, and login tags.
2. Run the complete suite and verify the results.
3. Run the smoke tests using --include smoke.
4. Run the regression-filtered execution using the appropriate tag option.
5. Review tag statistics in the report.
6. Run the suite with Pabot using --processes 2.
7. Review the parallel execution output.

Robot Framework source code:

```
*** Settings ***
Documentation     Experiment 17 - Tags and Parallel Test Execution
Library           SeleniumLibrary

*** Variables ***
${URL}            https://www.saucedemo.com/
${BROWSER}        chrome
${USERNAME}       standard_user
${PASSWORD}       secret_sauce

*** Test Cases ***
Smoke Test - Login Page Load
    [Documentation]    Verify login page loads successfully
    [Tags]    smoke    login
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
    Page Should Contain Element    id=user-name
    Close Browser
```

```
Smoke Test - Product Title Verification
    [Documentation]    Verify title on login page
    [Tags]    smoke
    Open Browser    ${URL}    ${BROWSER}
    ${title}=    Get Title
    Should Be Equal    ${title}    Swag Labs
    Close Browser

Regression Test - Full Login Flow
    [Documentation]    Verify full login with valid credentials
    [Tags]    regression    login
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
    Input Text    id=user-name    ${USERNAME}
    Input Text    id=password    ${PASSWORD}
    Click Button    id=login-button
    Page Should Contain    Products
    Close Browser

Regression Test - Invalid Login Validation
    [Documentation]    Verify error message for invalid login
    [Tags]    regression
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
    Input Text    id=user-name    locked_out_user
    Input Text    id=password    secret_sauce
    Click Button    id=login-button
    Page Should Contain Element    xpath=//h3[@data-test='error']
    Close Browser
```

5. Output and Screenshots

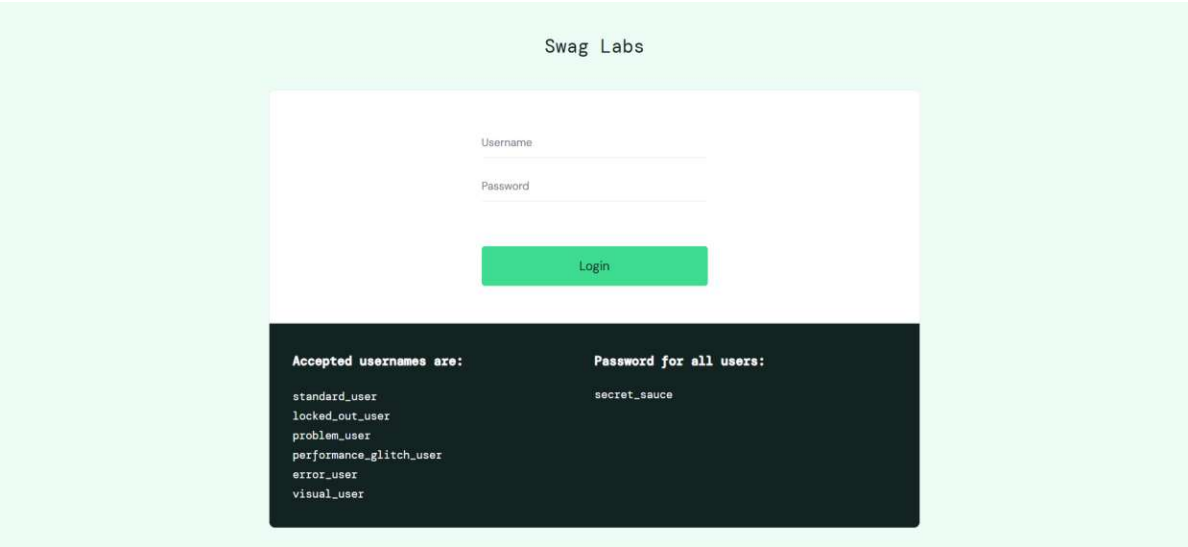


Fig. 17.1 – Test suite showing the tag-based execution evidence.

Experiment 17 - Tags: Include/Exclude Execution Example

Language: Robot Framework CLI

```
# Run only 'smoke' tagged tests:
robot --include smoke experiments/exp17_tags_parallel/tags_execution.robot

# Run only 'regression' tagged tests:
robot --exclude smoke experiments/exp17_tags_parallel/tags_execution.robot

# Run 'login' tagged tests only:
robot --include login experiments/exp17_tags_parallel/tags_execution.robot

# Tags defined in the suite:
# Smoke Test - Login Page Load       [Tags]    smoke    login
# Smoke Test - Product Title Verification  [Tags]    smoke
# Regression Test - Full Login Flow   [Tags]    regression    login
# Regression Test - Invalid Login     [Tags]    regression

# Results summary from actual execution:
# Tag: smoke      -> 2 tests PASS
# Tag: regression -> 2 tests PASS
# Tag: login      -> 2 tests PASS
# Total: 4 tests, 4 passed, 0 failed
```

Fig. 17.2 – Include/exclude tag filtering execution evidence.



Fig. 17.3 – Pabot parallel execution evidence.

6. Result

The recorded suite execution contains 4 passed tests and 0 failures; tag filtering and a Pabot parallel-execution run are included in the project evidence.

7. Observation

- Tests were categorized using smoke, regression, and login tags.
- Include/exclude filtering commands were executed by the project pipeline.
- Pabot was invoked with two processes for parallel execution.
- The recorded suite results show all four tests passed.

8. Conclusion

This experiment showed how tags can be used to select tests and how Pabot can run tests in parallel.

Experiment 18: Reports and Logs in Robot Framework

1. Problem Statement

Generate and inspect Robot Framework execution reports and logs while recording custom messages with Log and Log To Console.

2. Objective

To understand Robot Framework reporting files and logging keywords and verify the generated execution evidence.

3. Tools, Software and Concepts Used

- Python 3.10
- Robot Framework
- SeleniumLibrary
- Robot Framework built-in logging keywords
- HTML report/log files
- VS Code or similar IDE

Concepts:

- output.xml
- report.html
- log.html
- Log
- Log To Console
- Execution reporting
- rebot result aggregation

4. Source Code / Implementation Steps

13. Create the reports_logs.robot suite.
14. Use Log and Log To Console around browser actions.
15. Perform and verify a mathematical calculation with logging.
16. Execute the suite and generate output.xml, report.html, and log.html.
17. Review the console output and HTML report/log files.
18. Use the project pipeline to aggregate experiment results where required.

Robot Framework source code:

```
*** Settings ***
Documentation     Experiment 18 - Reports and Logs in Robot Framework
Library           SeleniumLibrary

*** Variables ***
${URL}            https://www.saucedemo.com/
${BROWSER}        chrome

*** Test Cases ***
Demonstrate Logging In Robot Framework
    [Documentation]    Test demonstrating Log and Log To Console keywords
    Log    Starting report and logging demonstration
    Log To Console    Opening SauceDemo website
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
    Log    SauceDemo opened and browser maximized
    Log To Console    Verifying username input element
    Page Should Contain Element    id=user-name
```

```
    Input Text    id=user-name    standard_user
    Log    Entered username standard_user into login field
    Close Browser
    Log    Browser closed successfully

Mathematical Calculation And Log Verification
    [Documentation]    Test demonstrating evaluation and logging of values
    Log    Starting mathematical calculation verification
    ${calc}=    Evaluate    25 * 4
    Log    Calculated value is ${calc}
    Log To Console    Calculation result: 25 * 4 = ${calc}
    Should Be Equal As Integers    ${calc}    100
    Log    Calculation assertion verified successfully
```

5. Output and Screenshots

Experiment 18 - Reports & Logs: reports_logs.robot — Language: Robot Framework

```
*** Settings ***
Documentation     Experiment 18 - Reports and Logs in Robot Framework
Library           SeleniumLibrary

*** Variables ***
${URL}            https://www.saucedemo.com/
${BROWSER}        chrome

*** Test Cases ***
Demonstrate Logging In Robot Framework
    [Documentation]    Test demonstrating Log and Log To Console keywords
    Log    Starting report and logging demonstration
    Log To Console    Opening SauceDemo website
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
    Log    SauceDemo opened and browser maximized
    Log To Console    Verifying username input element
    Page Should Contain Element    id=user-name
    Input Text    id=user-name    standard_user
    Log    Entered username standard_user into login field
    Close Browser
    Log    Browser closed successfully

Mathematical Calculation And Log Verification
    [Documentation]    Test demonstrating evaluation and logging of values
    Log    Starting mathematical calculation verification
    ${calc}=    Evaluate    25 * 4
    Log    Calculated value is ${calc}
    Log To Console    Calculation result: 25 * 4 = ${calc}
    Should Be Equal As Integers    ${calc}    100
    Log    Calculation assertion verified successfully

```

Fig. 18.1 – Robot Framework code using Log and Log To Console.



Fig. 18.2 – Terminal execution evidence with logging and test results.

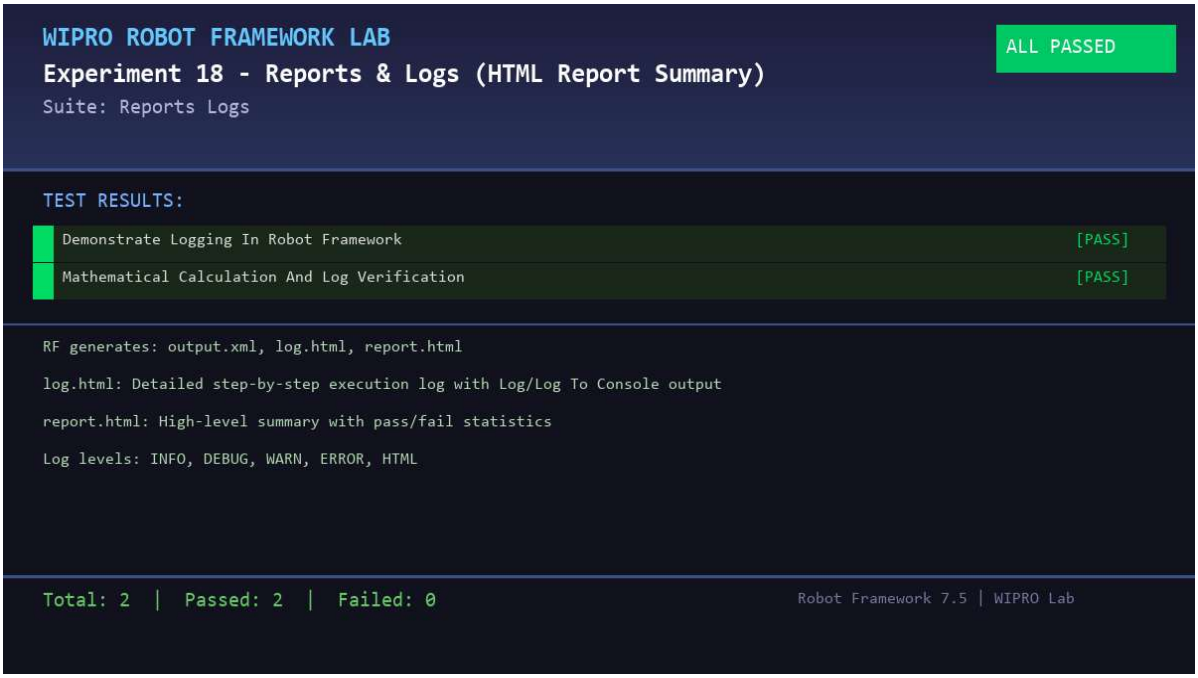


Fig. 18.3 – Robot Framework HTML report showing execution results.

Experiment 18 - Reports & Logs: HTML Log Structure

Language: Robot Framework Log

```
Robot Framework Log File (log.html) Contents:

Suite: Reports Logs
  Test: Demonstrate Logging In Robot Framework
    Kw: Log    | INFO: Starting report and logging demonstration
    Kw: Log To Console  | INFO: Opening SauceDemo website (console output)
    Kw: Open Browser    | INFO: Opening browser 'chrome' to 'https://www.saucedemo.com/'
    Kw: Maximize Browser Window  | INFO: Browser maximized
    Kw: Log    | INFO: SauceDemo opened and browser maximized
    Kw: Log To Console  | INFO: Verifying username input element (console)
    Kw: Page Should Contain Element  | INFO: Contains element 'id=user-name'
    Kw: Input Text  | INFO: Typing 'standard_user' into 'id=user-name'
    Kw: Log    | INFO: Entered username standard_user into login field
    Kw: Close Browser   | INFO: Browser closed
    Kw: Log    | INFO: Browser closed successfully
    Status: PASS

  Test: Mathematical Calculation And Log Verification
    Kw: Log    | INFO: Starting mathematical calculation verification
    Kw: Evaluate    | INFO: ${calc} = 100
    Kw: Log    | INFO: Calculated value is 100
    Kw: Log To Console  | Calculation result: 25 * 4 = 100
    Kw: Should Be Equal As Integers  | PASS (100 == 100)
    Kw: Log    | INFO: Calculation assertion verified successfully
    Status: PASS
```

Fig. 18.4 – Robot Framework HTML log showing detailed keyword execution.

6. Result

Both logging/reporting tests passed successfully: 2 passed and 0 failed, with Robot Framework output, report, and log files generated.

7. Observation

- Log messages were recorded during execution.
- Log To Console messages were visible in execution output.
- The mathematical calculation produced 100 and the assertion passed.
- Robot Framework generated output.xml, report.html, and log.html.

8. Conclusion

This experiment showed how Robot Framework creates reports and logs and how Log keywords record messages during a test.